CENTRO INTEGRADO DE F.P.
PONFERRADA

MÓDULO PROYECTO

CURSO 2025__- 2026__

TITULO: MARYSHOPPING

ALUMNO: MARISOL VARELA CABADO

CICLO FORMATIVO: DESARROLLO DE APLICACIONES WEB(DAW)

TUTOR: BELKIS DÍAZ LAZURICA

RESUMEN

ÍNDICE

RESUMEN....2
ÍNDICE DE FIGURAS....4
Proyecto_MARYSHOPPING....5
1. Introducción....5
1.1. Descripción del proyecto y estado del arte....5
1.2. Objetivos....5
2. Análisis, diseño y desarrollo....5
2.1. Estimación de costes. Tiempo invertido. Justificación....5
2.2. Tecnologías y herramientas utilizadas (nombrarlas y pequeña descripción, justificación de selección)....5
2.3. Diseño de interfaz (diseño general de pantallas, estructura, paleta de colores, facilidades de uso, elementos destacados, etc.)....5
2.4. Concepción de la base de datos (MER, diagrama lógico de tablas, etc.) 5
2.5. Diseño y arquitectura de la solución (módulos, patrones utilizados en la programación, diseño de clases (si procede), ayudas concebidas, manipulación de errores, etc.)....5
2.6. Principales dificultades encontradas y cómo se han afrontado....5
2.7. Despliegue (dónde está alojada, cómo acceder (usuarios, contraseñas), qué se necesita, etc.)....5
2.8. Defensa arquitectura no database first....5
3. Conclusiones y recomendaciones....6
4. Bibliografía y webgrafía....6

ÍNDICE DE FIGURAS

Figura 1. Estructura anatómica de la madera de coníferas.....5

PROYECTO_MARYSHOPPING

1. Introducción

1.1. Descripción del proyecto y estado del arte

1.2. Objetivos.

OBJETIVO GENERAL

El objetivo principal de este proyecto es desarrollar el backend de una aplicación de comercio electrónico aplicando arquitecturas y patrones de diseño de software ampliamente adoptados en la industria (DDD, Vertical Slice Architecture, Monolito Modular), y documentar las decisiones tomadas de forma que sirva como referencia de aprendizaje y como demostración de las competencias adquiridas durante el ciclo formativo DAW.

OBJETIVOS ESPECÍFICOS

1. Implementar una API REST funcional con Spring Boot que cubra las operaciones necesarias para gestionar una tienda online:

- CRUD completo del catálogo (categorías, proveedores, productos, imágenes).

- CRUD completo de clientes con gestión de direcciones (facturación y envío).

- CRUD completo de pedidos y líneas de pedido.

- Consultas de reporting (productos más vendidos, clientes con mayor gasto).

2. Aplicar Domain-Driven Design (DDD) en los módulos principales:

- Definir entidades de dominio con value objects (Address, Email, etc.).

- Separar la capa de dominio del resto de capas.

- Usar Vertical Slice Architecture para organizar el código por caso de uso.

3. Organizar el sistema en módulos (bounded contexts) independientes:

- catalog, customers, orders, reports.

- Cada módulo con su propio esquema de base de datos PostgreSQL.

- Sin claves foráneas entre módulos (referencias lógicas por UUID).

4. Gestionar las migraciones de base de datos de forma controlada:

- Usar Flyway con configuración multi-módulo personalizada.

- Una instancia de Flyway por módulo, apuntando a su propio schema.

5. Incluir un módulo de reporting diferenciado (reports):

- Sin capa de dominio, siguiendo el principio CQRS para lectura.

- Tabla denormalizada como read model para consultas agregadas eficientes.

6. Documentar el proyecto de forma académica:

- Justificar las decisiones de diseño con fuentes reconocidas.

- Generar diagramas (flujo, casos de uso, secuencia, MER).

- Defender el enfoque adoptado frente al enfoque "Database-First".

7. Preparar el sistema para despliegue con Docker:

- Incluir Dockerfile para contenerizar la aplicación.

- Configurar la aplicación para conectarse a PostgreSQL.

8. Diseñar e implementar un sistema de seguridad RBAC con JWT:

- Definir los roles del sistema (ROLE_ANONYMOUS, ROLE_CUSTOMER, ROLE_ADMIN).

- Especificar los permisos de cada rol sobre cada endpoint de la API.

- Implementar autenticación stateless con JWT mediante Spring Security.

- Aplicar restricciones de propiedad (un cliente solo accede a sus propios datos).

9. Desarrollar el frontend de la aplicación con Nuxt 4 (Vue 3):

- Implementar la interfaz de usuario con Vuetify 3 (Material Design).

- Catálogo de productos, carrito de compra, gestión de pedidos y perfil de usuario.

- Panel de administración (CRUD de categorías, proveedores, productos y clientes).

- Gestión de estado con Pinia.

OBJETIVOS DE APRENDIZAJE PERSONAL

- Comprender y aplicar los principios de DDD en un proyecto real.

- Entender qué es un bounded context y cómo separar módulos de forma efectiva, afrontando la complejidad del aislamiento real entre contextos.

- Aprender a usar Spring Boot, Spring Data JPA, Flyway y Lombok en combinación.

- Aprovechar las funcionalidades de Java 25 que se usan en el proyecto (records y streams) para implementar los patrones arquitectónicos con menos código repetitivo y de forma más clara.

- Desarrollar la capacidad de justificar decisiones técnicas con argumentos sólidos, entendiendo las ventajas e inconvenientes de cada opción.

- Ganar experiencia con herramientas profesionales (Java 25, Maven, Docker, Postman, Kubernetes).

- Entender el valor de la arquitectura de software en un contexto donde la generación de código se automatiza cada vez más.

2. Análisis, diseño y desarrollo

2.1. Estimación de costes. Tiempo invertido. Justificación.

2.2. Tecnologías y herramientas utilizadas (nombrarlas y pequeña descripción, justificación de selección)

2.3. Diseño de interfaz (diseño general de pantallas, estructura, paleta de colores, facilidades de uso, elementos destacados, etc.).

2.4. Concepción de la base de datos

2.4.1. ARQUITECTURA DE BASE DE DATOS

La base de datos del proyecto es una única instancia de PostgreSQL con CUATRO schemas separados, uno por módulo (bounded context):

```
PostgreSQL: maryshopping
├── Schema: customers
│   └── Tabla: customer
├── Schema: catalog
│   ├── Tabla: category
│   ├── Tabla: provider
│   ├── Tabla: product
│   └── Tabla: product_image
├── Schema: orders
│   ├── Tabla: purchase_order
│   └── Tabla: order_line
└── Schema: reports
    └── Tabla: order_report
```

Regla fundamental: NO existen claves foráneas entre schemas. Las referencias entre módulos se hacen únicamente por UUID (identificador lógico), sin FK declarada.

2.4.2 MODELO LÓGICO:

2.4.2.1 Schema CUSTOMERS

Tabla: customers.customer

- id (UUID, PK): identificador único del cliente.
- first_name, last_name: nombre y apellidos.
- dni (TEXT, UNIQUE): documento de identidad.

- phone_number: teléfono de contacto.

- email_address (TEXT, UNIQUE): correo electrónico.

- billing_address_street, billing_address_city, billing_address_province,

  billing_address_postal_code, billing_address_country:

  dirección de facturación (5 columnas, embebida del value object Address).

-shipping_address_street, shipping_address_city, shipping_address_province,

  shipping_address_postal_code, shipping_address_country:

  dirección de envío (5 columnas, embebida del value object Address).

Decisión de diseño: Address no es una tabla separada. Se almacena como columnas embebidas. Esto simplifica las consultas y evita joins innecesarios. El value object Address existe en el código (shared_kernel) pero no como tabla.

2.4.2.2 Schema CATALOG

Tabla: catalog.category

- id (UUID, PK): identificador de la categoría.

- name (TEXT, UNIQUE): nombre de la categoría.

Tabla: catalog.provider

- id (UUID, PK): identificador del proveedor.

- name (TEXT, UNIQUE): nombre del proveedor.

Tabla: catalog.product

- id (UUID, PK): identificador del producto.

- category_id (UUID, FK → catalog.category.id): categoría del producto.

- provider_id (UUID, FK → catalog.provider.id): proveedor del producto.

-product_provider_ref (TEXT): referencia del producto en el catálogo del proveedor.

- name (TEXT): nombre del producto.

- description (TEXT): descripción del producto.

- price (NUMERIC): precio del producto.

Tabla: catalog.product_image

- id (UUID, PK): identificador de la imagen.

- product_id (UUID, FK → catalog.product.id): producto al que pertenece la imagen.

- image_mime_type (TEXT): tipo MIME de la imagen (ej: image/jpeg).

- image_order (INTEGER): orden de visualización.

- is_primary (BOOLEAN): indica si es la imagen principal.

- image (BYTEA): datos binarios de la imagen.

Relaciones DENTRO del schema catalog:

- product.category_id → category.id (FK con integridad referencial)

- product.provider_id → provider.id (FK con integridad referencial)

- product_image.product_id → product.id (FK con integridad referencial)

2.4.2.3 Schema ORDERS

Tabla: orders.purchase_order

- id (UUID, PK): identificador del pedido.

- customer_id (UUID): ID del cliente. SIN FK (referencia lógica a customers.customer).

- order_date (BIGINT): fecha del pedido en formato epoch milliseconds.

- shipping_street, shipping_city, shipping_province,

  shipping_postal_code, shipping_country:

  dirección de envío del pedido (embebida, igual que en customers).

Tabla: orders.order_line

- order_line_id (UUID, PK): identificador de la línea de pedido.

- order_id (UUID, FK → orders.purchase_order.id): pedido al que pertenece.

- product_id (UUID): ID del producto. SIN FK (referencia lógica a catalog.product).

- product_name (VARCHAR): nombre del producto en el momento del pedido (snapshot).

- product_provider_reference (VARCHAR): referencia del proveedor (snapshot).

- category_name (VARCHAR): nombre de la categoría (snapshot).

- provider_name (VARCHAR): nombre del proveedor (snapshot).

- quantity (INTEGER): cantidad pedida.

- unit_price (NUMERIC): precio unitario en el momento del pedido (snapshot).

Nota sobre snapshots: los campos de product_name, category_name, etc. son snapshots del momento del pedido. Si el producto cambia de nombre o precio después, el pedido histórico mantiene los valores originales. Esta es una decisión de diseño habitual en pedidos de e-commerce: un pedido debe reflejar los datos vigentes en el momento de la compra, no los datos actuales. Si un cliente cambia su dirección, los pedidos pasados no deben mostrar la nueva dirección, sino la dirección a la que se envió el paquete originalmente.

Relaciones DENTRO del schema orders:

- order_line.order_id → purchase_order.id (FK con integridad referencial)

2.4.2.4 Schema REPORTS

Tabla: reports.order_report

- order_line_id (UUID, PK)

- order_id (UUID)

- order_date (BIGINT)

- customer_id, customer_first_name, customer_last_name, customer_dni

- shipping_street, shipping_city, shipping_province, shipping_postal_code, shipping_country

- product_id, product_name, product_provider_reference, category_name, provider_name

- quantity (INTEGER)

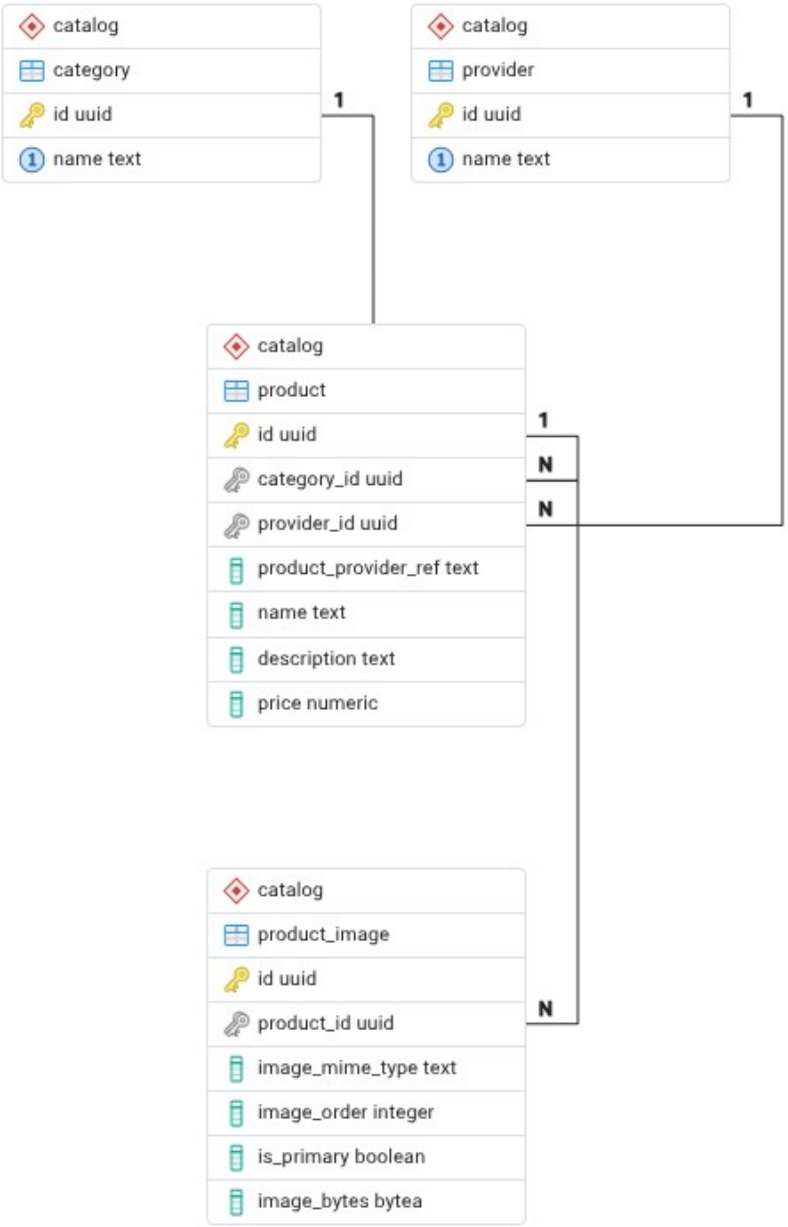
- unit_price (NUMERIC)

- sub_total (NUMERIC): quantity × unit_price

- total_amount_due (NUMERIC): total del pedido completo

Esta tabla es un "read model" denormalizado: contiene todos los datos necesarios para los informes en una sola tabla, evitando joins entre schemas. Es el complemento del principio CQRS aplicado en este proyecto.

2.4.3. DIAGRAMA DE CLASES



customers

customer

id uuid

first_name text

last_name text

dni text

phone_number text

email_address text

billing_address_street text

billing_address_city text

billing_address_province text

billing_address_postal_code text

billing_address_country text

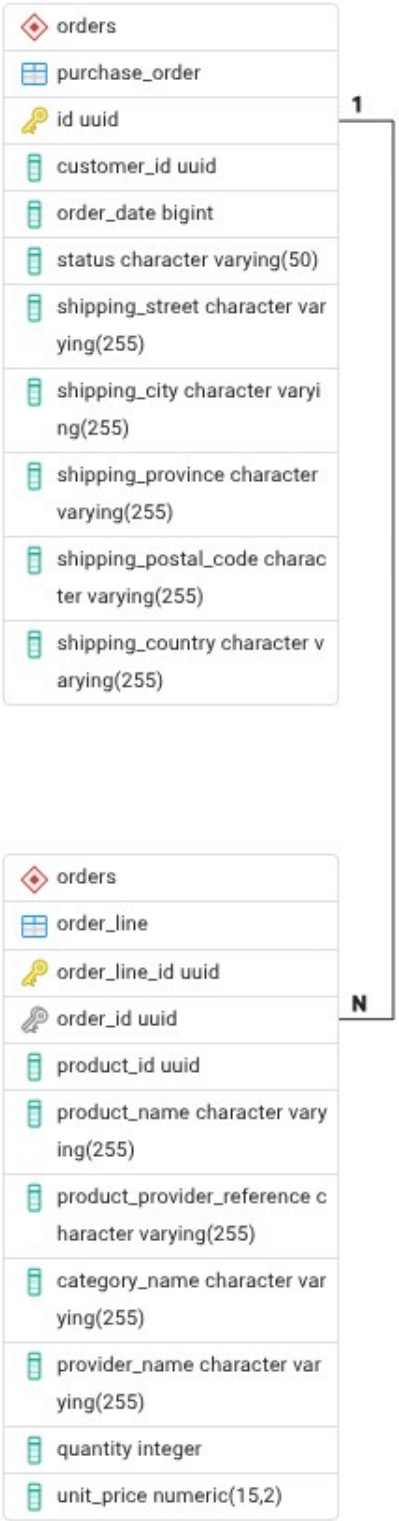
shipping_address_street text

shipping_address_city text

shipping_address_province text

shipping_address_postal_code text

shipping_address_country text


orders
purchase_order
id uuid
customer_id uuid
order_date bigint
status character varying(50)
shipping_street character varying(255)
shipping_city character varying(255)
shipping_province character varying(255)
shipping_postal_code character varying(255)
shipping_country character varying(255)
1
orders
order_line
order_line_id uuid
order_id uuid
product_id uuid
product_name character varying(255)
product_provider_reference character varying(255)
category_name character varying(255)
provider_name character varying(255)
quantity integer
unit_price numeric(15,2)
N

reports
order_report
order_id uuid
order_date bigint
customer_id uuid
customer_first_name character varying(255)
customer_last_name character varying(255)
customer_dni character varying(255)
shipping_street character varying(255)
shipping_city character varying(255)
shipping_province character varying(255)
shipping_postal_code character varying(255)
shipping_country character varying(255)
order_line_id uuid
product_id uuid
product_name character varying(255)
product_provider_reference character varying(255)
category_name character varying(255)
provider_name character varying(255)
quantity integer
unit_price numeric(15,2)
sub_total numeric(15,2)
total_amount_due numeric(15,2)

2.4.4. GESTIÓN DE MIGRACIONES CON FLYWAY

Las migraciones de base de datos se gestionan con Flyway, con una configuración personalizada multi-módulo:

- Cada módulo tiene su propio conjunto de scripts SQL en:

  src/main/resources/{módulo}/db/migration/postgresql/

- Los scripts siguen la convención de nombres de Flyway:

  V{version}__{descripcion}.sql

  Ejemplo: V1_0_0__init_customers.sql

- FlywayConfig.java instancia un objeto Flyway por cada módulo, configurado para apuntar al schema correspondiente.

- MultiModuleFlywayMigrationStrategyConfig.java asegura que las migraciones de todos los módulos se ejecutan en el orden correcto al arrancar la aplicación.

Ventaja: si se añade un nuevo módulo o se modifica el schema de uno existente, solo es necesario añadir un nuevo script de migración en la carpeta correspondiente.Los demás módulos no se ven afectados.

2.4.5. JUSTIFICACIÓN DEL DISEÑO DE BASE DE DATOS

El diseño de base de datos con schemas separados y sin FKs globales es una consecuencia directa de la arquitectura de módulos (bounded contexts) del proyecto. En resumen:

- La integridad referencial DENTRO de cada módulo sí se garantiza con FKs.

- La consistencia ENTRE módulos se garantiza en la capa de aplicación.

- Los schemas separados permiten que cada módulo evolucione de forma independiente.

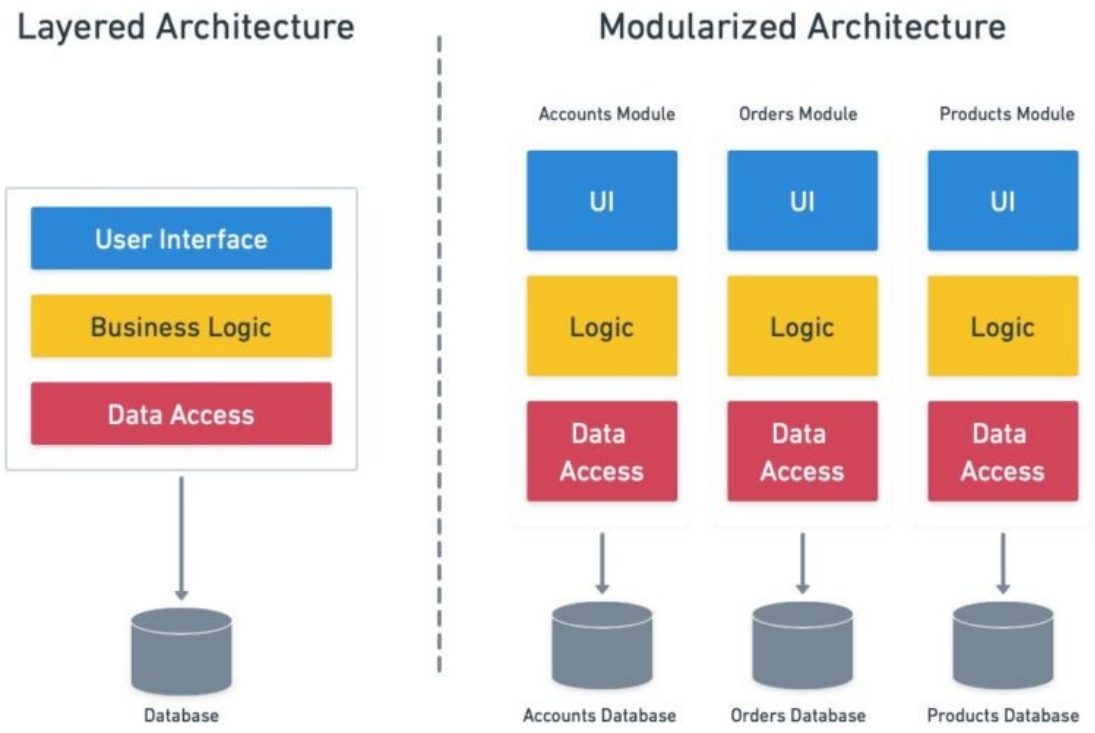
- El módulo reports tiene su propio esquema optimizado para consultas de reporting, siguiendo el principio CQRS.

2.5. Diseño y arquitectura de la solución

2.5.1. VISIÓN GENERAL DE LA ARQUITECTURA

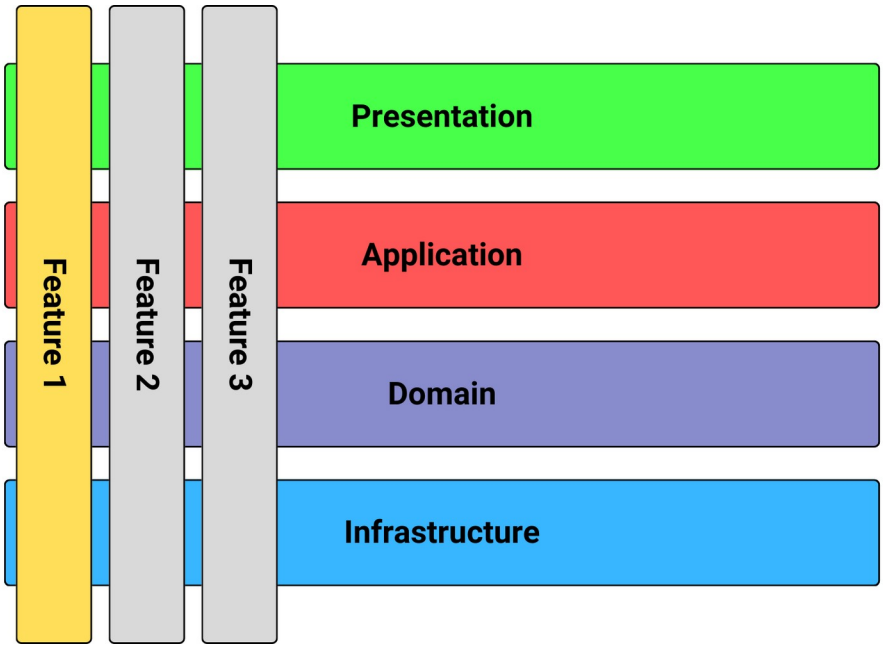
Este proyecto aplica dos formas de organizar el código que se combinan entre sí:

a) MONOLITO MODULAR: una única aplicación (un solo fichero .jar, un solo despliegue) dividida por dentro en módulos bien separados. Es más sencillo que tener varios servicios separados (microservicios), pero aplica la misma idea de mantener cada parte independiente de las demás.



Fuente: https://semaphore.io/blog/monolith-microservices

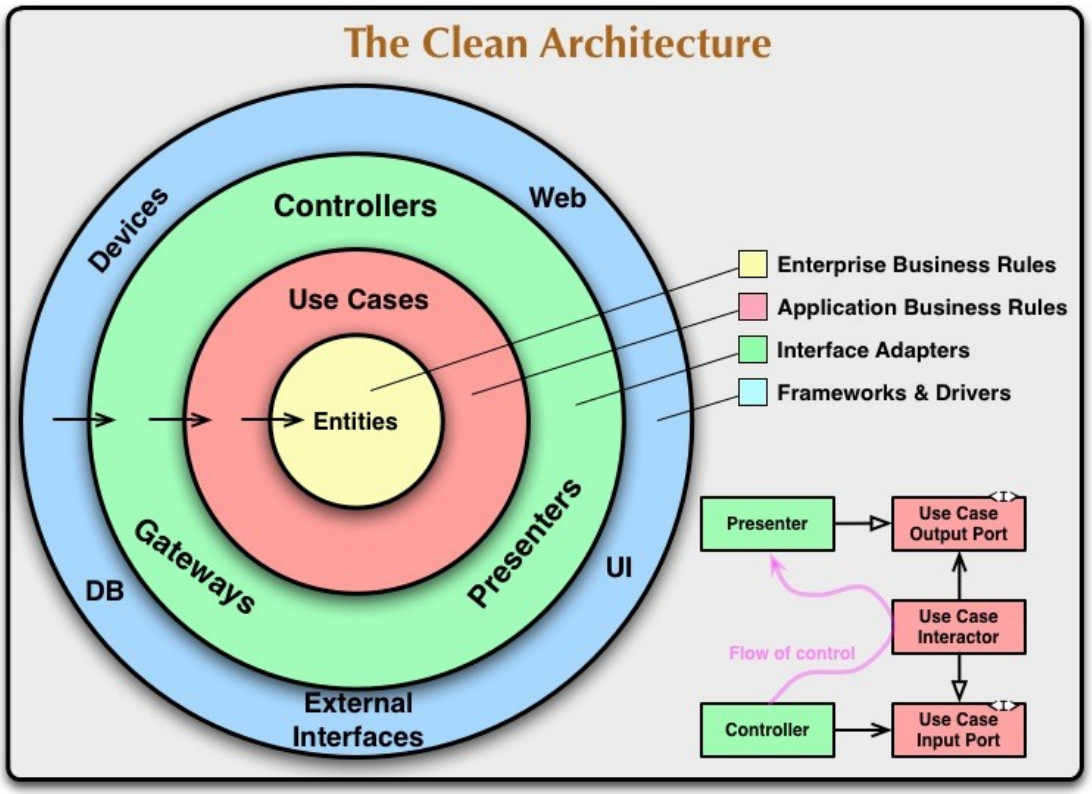
b) VERTICAL SLICE ARCHITECTURE: el código se organiza por caso de uso (lo que en el proyecto llamamos "rebanadas verticales"), no por capas técnicas (una carpeta para controllers, otra para services...). Cada caso de uso tiene todo lo que necesita en su propia carpeta.



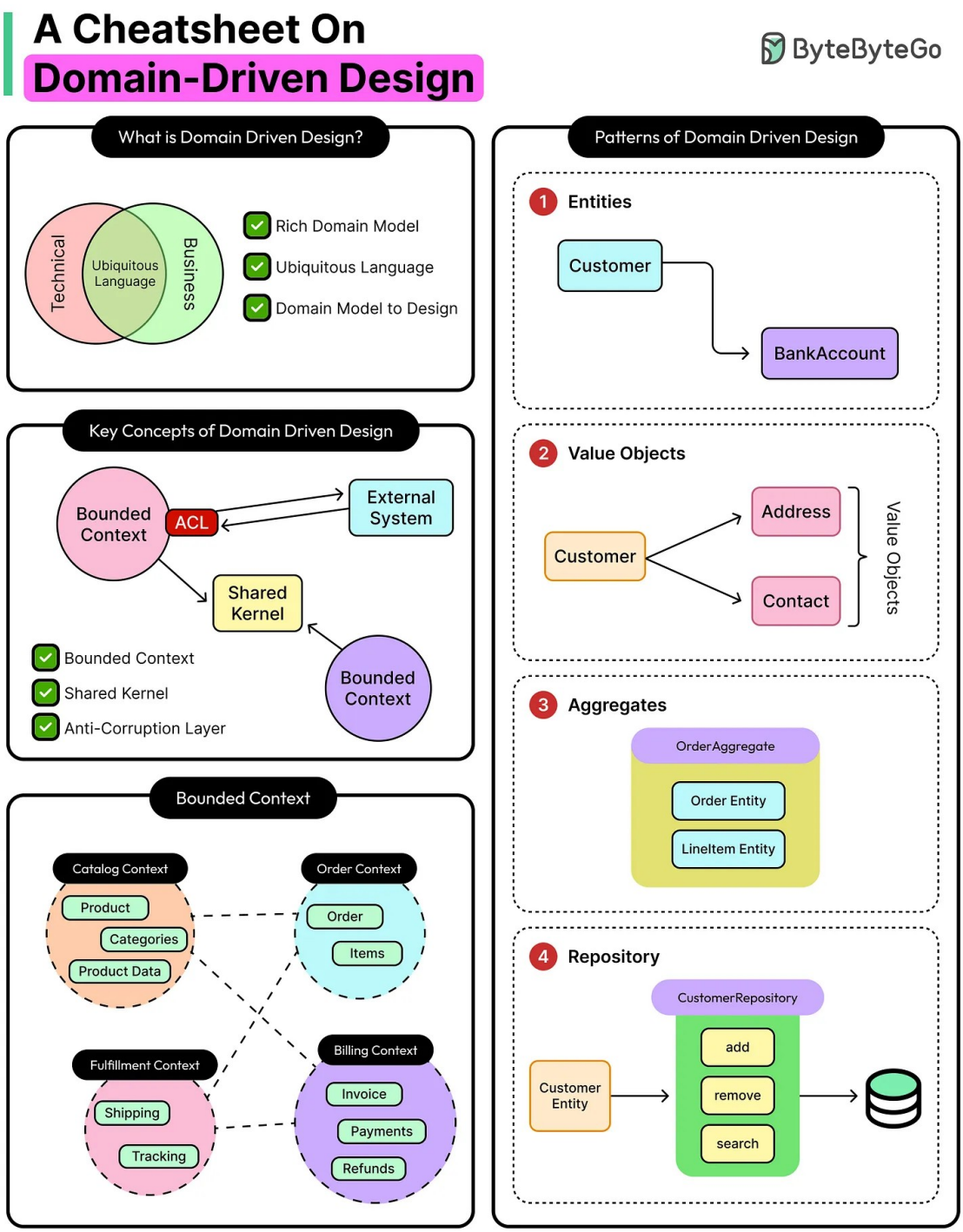
Fuente: https://www.milanjovanovic.tech/blog/vertical-slice-architecture

Estos se complementan con:

- Clean Architecture y Domain-Driven Design (DDD): una forma de diseñar el código que pone en el centro las reglas del negocio (el "dominio"), en lugar de las tablas de la base de datos.

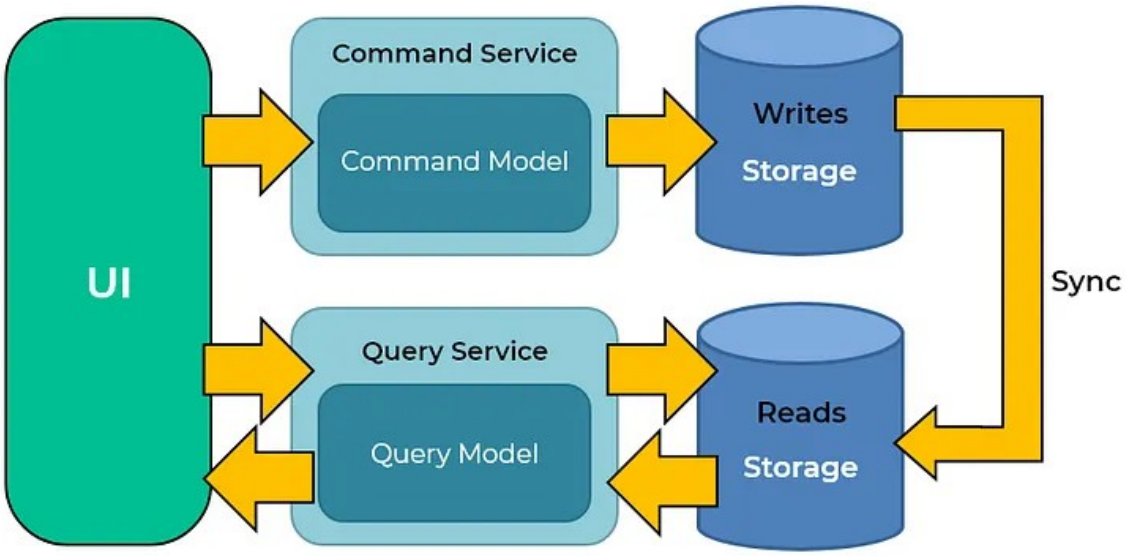


Fuente: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html



Fuente: https://blog.bytebytego.com/p/domain-driven-design-ddd-demystified

- CQRS parcial: el módulo reports separa el modelo de escritura (donde se guardan datos) del modelo de lectura (donde se consultan informes). "Parcial" significa que solo se aplica en reports; el resto de módulos usan la misma tabla para leer y escribir.



Fuente: https://medium.com/@emer.kurbegovic/cqrs-software-architecture-pattern-the-good-the-bad-and-the-ugly-efe48e8dcd14

El backend expone una API REST que es consumida por un frontend desarrollado con Nuxt 4 .

2.5.2. ¿POR QUÉ SE ESCRIBE EL CÓDIGO EN INGLÉS?

Todo el código del proyecto (nombres de clases, variables, métodos, paquetes) está en inglés, aunque la documentación académica está en español. Las razones son:

a) Es el estándar en la industria del software. Los tutoriales, las anotaciones de Spring, los mensajes de error y todas las librerías están en inglés. Escribir código mezclando idiomas (CreateCategoriaController, getProductoById) crearía confusiones.

b) La letra ñ y los acentos causan problemas. Muchas herramientas (terminales, codificaciones de texto, URLs) no manejan bien los caracteres especiales del español. Un nombre de variable como "año" puede dar problemas de codificación.

c) El software se desarrolla cada vez más en equipos internacionales. Usar inglés facilita que cualquier persona pueda leer y contribuir al código, independientemente de su idioma nativo.

d) Toda la documentación técnica de referencia (Spring, Java, PostgreSQL) está en inglés. Usar los mismos términos en el código evita confusiones.

2.5.3. PATRÓN VERTICAL SLICE — ESTRUCTURA DE UN CASO DE USO

Flujo de una petición:

1. El Controller recibe la petición HTTP con un Request (DTO JSON).

2. El Controller mapea el Request a un Command.

3. El Controller llama al Service con el Command.

4. El Service construye el objeto de dominio (validando y creando value objects).

5. El Service llama al Repository para persistir.

6. El Repository mapea el dominio a una entidad JPA y guarda en base de datos.

7. El Service devuelve un Result al Controller.

8. El Controller mapea el Result a una Response (DTO JSON) y responde.

2.5.4. CAPAS Y RESPONSABILIDADES

Capa de infraestructura (infrastructure/):

- Controller: recibe la petición HTTP, delega en el Service y devuelve la respuesta HTTP.

- Request: record Java que representa el JSON de entrada.

- Response: record Java que representa el JSON de salida.

Capa de aplicación (application/):

- Service: coordina la lógica de flujo del caso de uso(orquestación). Construye el objeto de dominio, lo persiste a través del repositorio y devuelve el resultado.

- Command: record Java que contiene los datos necesarios para ejecutar el caso de uso. Es el "contrato" entre el controller y el service.

Capa de dominio (shared/domain dentro de cada módulo):

- Entidad de dominio: record Java que representa el concepto del negocio con todos sus value objects. No tiene anotaciones JPA.

- Value objects: tipos que encapsulan un valor del dominio (City, Street, etc.).

Capa de persistencia (shared/infrastructure/persistence dentro de cada módulo):

- Entidad JPA: clase Java anotada con @Entity que mapea a la tabla de base de datos. Tiene un método estático fromDomain() para construirse desde el dominio.

- Spring Repository: interfaz que extiende JpaRepository y que Spring implementa automáticamente.

2.5.5. PATRONES DE DISEÑO UTILIZADOS

a) Builder Pattern (Patrón Constructor — Lombok @Builder):

Permite construir objetos paso a paso, indicando cada campo por su nombre. Esto evita confusiones cuando un objeto tiene muchos campos, porque no dependemos del orden de los parámetros.

Sin Builder: new Customer("María", "González", "12345678A", "634567890"...)

Con Builder: Customer.builder().firstName("María").dni("12345678A").build()

En el primer caso, si confundimos el orden (todos son String), el compilador no avisa. En el segundo, cada campo va con su nombre. En este proyecto, todos los Commands y las entidades JPA usan @Builder de Lombok.

b) Record (Java):

Un record es un tipo especial de clase que Java introdujo a partir de la versión 16. Es inmutable: una vez creado, no se puede cambiar. Java genera automáticamente los métodos equals(), hashCode() y toString(). Antes de los records, para un objeto con 5 campos había que escribir unas 40-50 líneas solo para el constructor, getters, equals y hashCode. Con un record es una sola línea:

public record City(String value) {}

Se usa para DTOs, Commands, Requests, Responses y Value Objects.

c) Repository Pattern (Patrón Repositorio):

Separa la lógica de negocio del acceso a datos. En el código, el repositorio se define como una interfaz (CustomerSpringRepository que extiende JpaRepository). Spring Data JPA genera la implementación automáticamente cuando la aplicación arranca: detecta que es una interfaz que extiende JpaRepository y crea internamente una clase que implementa todos los métodos heredados (findById, save, deleteById...). El programador nunca ve esa clase, pero existe y funciona.

d) Factory Method (Método Factoría — fromDomain()):

Las entidades JPA tienen un método estático fromDomain(Customer customer) que construye la entidad JPA a partir del objeto de dominio. Esto centraliza la lógica de conversión en un solo lugar.

e) Inyección de Dependencias (Dependency Injection) por Constructor:

Este es uno de los conceptos más importantes de Spring y del proyecto. Vamos a explicarlo paso a paso.

EL PROBLEMA:

Imaginemos que el GetCustomerService necesita acceder a la base de datos. La forma más directa sería crear el repositorio dentro del propio servicio:

```
public class GetCustomerService {
    public GetCustomerResult execute(UUID id) {
        CustomerSpringRepository repo = new CustomerSpringRepository();
        // ...
    }
}
```

Esto tiene un problema: el servicio está creando su propia dependencia. Si queremos cambiar el repositorio, hay que entrar en el servicio y cambiar esa línea. Además, el servicio "sabe demasiado" sobre cómo se crea el repositorio.

LA SOLUCIÓN — INYECCIÓN DE DEPENDENCIAS:

En lugar de que el servicio cree sus dependencias, alguien se las "da" (se las "inyecta") desde fuera. Ese "alguien" es Spring.

```
public class GetCustomerService {
    private final CustomerSpringRepository repository;
    // Spring llama a este constructor y le pasa el repositorio ya creado
    public GetCustomerService(CustomerSpringRepository repository) {
        this.repository = repository;
    }
}
```

El servicio solo dice "necesito un repositorio" (lo declara como parámetro del constructor), y Spring se encarga de crearlo y pasárselo.

¿QUÉ ES INVERSIÓN DE CONTROL (IoC)?

IoC (Inversion of Control) significa que el control sobre la creación de objetos ya no lo tiene el programador, sino el framework (Spring). Sin IoC, el programador decide cuándo y cómo crear cada objeto con "new". Con IoC, Spring se encarga de todo: crea los objetos, los conecta entre sí y los destruye cuando ya no se necesitan.

NOTA SOBRE @Autowired:

En muchos tutoriales de Spring se ve la anotación @Autowired en los campos:

```
@Autowired
private CustomerSpringRepository repository;
```

Esto funciona, pero la práctica recomendada (y la que se usa en este proyecto) es la inyección por constructor. La razón es la claridad: al mirar el constructor se ve de un vistazo de qué depende la clase. Con @Autowired en campos, las dependencias están "escondidas" por la clase.

La documentación oficial de Spring recomienda la inyección por constructor.

f) Value Object (DDD):

Un Value Object es un tipo que encapsula un valor del dominio y se compara por su contenido, no por su posición en memoria. Dos objetos City ("Madrid") son iguales porque contienen el mismo valor, aunque sean dos objetos diferentes. En este proyecto, City, Street, PostalCode, Province, Country y Address son value objects.

Al estar implementados como records de Java, la inmutabilidad (no se pueden cambiar después de crearse) y la comparación por valor vienen "gratis": Java las genera automáticamente sin que haya que escribir código.

2.5.6. SHARED KERNEL (NÚCLEO COMPARTIDO DE DDD)

En Domain-Driven Design, el Shared Kernel es un acuerdo entre varios módulos para compartir un conjunto pequeño y bien definido de código. En lugar de copiar los mismos tipos en cada módulo, se ponen en un sitio central que todos usan.

Referencia: Evans, E. (2003). Domain-Driven Design. Addison-Wesley.

Capítulo 14: "Maintaining Model Integrity".

En este proyecto, el shared_kernel contiene:

- Address (record): value object que representa una dirección completa. Compuesto por: City, Country, PostalCode, Province, Street.

- Excepciones comunes: EntityNotFoundException, BusinessRuleException, DuplicatedEntityException .

Es importante que el shared_kernel sea lo más pequeño posible: solo deben ir ahí los tipos que realmente necesitan compartirse.

2.5.7. COMANDOS Y QUERIES

En el proyecto se usan dos tipos de objetos para comunicar datos entre capas:

a) Command (Comando):

Lleva los datos necesarios para una operación de escritura (crear, actualizar, eliminar). Es un record con @Builder de Lombok. Los Commands NO contienen lógica. El mapeo a objetos de dominio se hace siempre en el Service (método fromCommandToDomain).

b) Query (Consulta):

Lleva los datos necesarios para una operación de lectura.

2.5.8. GESTIÓN CENTRALIZADA DE ERRORES: GLOBALEXCEPTIONHANDLER

En lugar de manejar los errores en cada controller, el proyecto usa un manejador global con @RestControllerAdvice. Intercepta las excepciones de cualquier controller y las convierte en respuestas HTTP apropiadas:

- EntityNotFoundException → HTTP 400 (Bad Request)
- BusinessRuleException / DuplicatedEntityException → HTTP 409 (Conflict)
- Exception (cualquier otra) → HTTP 500 (Internal Server Error)

2.5.9. JUSTIFICACIÓN DEL USO DE UUID

Todas las claves primarias son UUIDs generados por la aplicación, en lugar deIDs autoincrementales generados por la base de datos (id=1, id=2, id=3...).

¿Por qué?

a) El objeto existe antes de guardarse:

Con un UUID, el objeto tiene su ID desde que se crea en el código, antes de ir a la base de datos. Con un ID autoincremental, hay que esperar al INSERT para que la base de datos asigne el número. Es como necesitar un número de ticket antes de llegar a la ventanilla: con UUID te generas tu propio número único antes de llegar, en lugar de esperar a que te lo den.

b) No hay colisiones entre instancias:

Si hay varias copias del servidor funcionando a la vez (ver sección 18), un ID autoincremental podría repetirse. Un UUID es único sin necesidad de coordinación.

Referencia: RFC 4122 — UUID. https://datatracker.ietf.org/doc/html/rfc4122

c) Seguridad:

Los IDs autoincrementales son predecibles. Si el último cliente es id=42, un atacante puede probar id=41. Los UUIDs no son predecibles.

Referencia: OWASP — IDOR.

d) Coherencia con DDD:

El ID es propiedad del objeto de dominio, no de la base de datos.

2.5.10. JUSTIFICACIÓN DEL USO DE LONG PARA FECHAS

Las fechas se almacenan como BIGINT (Long en Java): milisegundos desde el 1 de enero de 1970 UTC (Unix epoch).

a) Sin problemas de zona horaria: un número siempre significa lo mismo.

b) Compatible con JavaScript: Date.now() devuelve milisegundos.

c) Comparar fechas es comparar números: fecha1 < fecha2.

d) Funciona igual en todas las bases de datos.

2.5.11. SNAPSHOTS: COPIAS INMUTABLES DE DATOS

Un snapshot es una copia de datos que se guarda tal como eran en un momento concreto. Es como una foto: aunque cambie después, la foto sigue mostrando cómo era cuando se hizo.

Ejemplo: un cliente pide la "Camiseta Azul" a 19,99 €. Dos semanas después, el precio sube a 24,99 €. El pedido del cliente debe seguir mostrando 19,99 €.

Referencia: Vernon, V. (2013). Implementing Domain-Driven Design.

Addison-Wesley. Capítulo 7: "Aggregates".

2.5.12. REDUNDANCIA DELIBERADA DE DATOS EN DDD

El mismo dato puede existir en varios sitios:

- catalog.product → nombre actual (editable).

- orders.order_line → nombre cuando se hizo el pedido (snapshot).

- reports.order_report → nombre en el informe (snapshot).

En DDD, esto es una decisión consciente. Cada módulo tiene su propia visión del dato y es completamente independiente.

Referencia: Evans, E. (2003). DDD. Capítulo 14: "Maintaining Model Integrity".

2.5.13. PROGRAMACIÓN FUNCIONAL EN JAVA: LAMBDAS, METHOD REFERENCES Y STREAMS

Java, a partir de la versión 8, incorporó herramientas de programación funcional que este proyecto usa con frecuencia.

¿Qué es un Stream?

Un Stream es una forma de procesar listas de datos indicando qué hacer con ellos, en lugar de cómo hacerlo paso a paso. Es como una cinta transportadora: los datos van pasando y en cada estación se les aplica una operación (filtrar, transformar, agrupar...).

2.5.14. CONVENCIÓN DE NOMBRES: SNAKE_CASE EN PAQUETES

Los paquetes de los casos de uso usan snake_case (create_customer,get_all_orders). La convención habitual en Java es minúsculas sin separadores, pero la especificación no prohíbe guiones bajos. Se usa por legibilidad:

"create_customer" es más claro que "createcustomer".

Referencia: Java Language Specification, §6.1.

2.5.15. EL VALOR DE LA ARQUITECTURA EN LA ERA DE LA IA

Con herramientas de IA capaces de generar código, ¿sigue importando la arquitectura? La respuesta es que la arquitectura es lo que la IA no decide por sí misma: cómo dividir el sistema en módulos, qué datos pertenecen a cada uno, cuándo guardar un snapshot y cuándo referenciar el dato original, o cómo diseñar un sistema mantenible a largo plazo.

En un contexto donde generar código es cada vez más fácil, lo que aporta valor es saber qué arquitectura aplicar y por qué.

2.6. Principales dificultades encontradas y cómo se han afrontado.

2.7. Despliegue (dónde está alojada, cómo acceder (usuarios, contraseñas), qué se necesita, etc.)

2.8. Defensa arquitectura no database first

NOTA PREVIA

En este apartado se justifica por qué se ha optado por una arquitectura modular con esquemas de base de datos separados por módulo (bounded context), en lugar del enfoque tradicional de diseñar primero la base de datos y construir el código alrededor de ese esquema (enfoque conocido como "Database-First").

No se trata de afirmar que el enfoque Database-First sea incorrecto en todos los casos. Es un enfoque que ha funcionado durante décadas y sigue siendo válido en determinados contextos. Lo que se argumenta aquí es que, para este proyecto y para el tipo de aplicaciones que se desarrollan hoy en día, el enfoque modular con Domain-First ofrece ventajas claras y bien documentadas en la literatura de ingeniería del software.

1. ¿QUÉ ES EL ENFOQUE DATABASE-FIRST Y QUÉ ALTERNATIVA SE PROPONE?

En el enfoque Database-First, el desarrollo comienza diseñando las tablas de la base de datos (el modelo entidad-relación completo), y después se construye el código de la aplicación alrededor de ese esquema. Todas las tablas viven en un único esquema compartido, conectadas entre sí por claves foráneas (Foreign Keys).

El enfoque alternativo que se aplica en este proyecto es el "Domain-First" o "Code-First modular":

- Se diseña primero el código que refleja las reglas del negocio (el dominio).

- La base de datos es un detalle de implementación que se adapta al dominio, no al revés.

- El sistema se divide en módulos independientes (bounded contexts), cada uno con su propio esquema de base de datos.

- La integridad referencial se mantiene DENTRO de cada módulo (con FKs normales), pero NO entre módulos (las referencias entre módulos son lógicas, por UUID).

1A. CONTEXTO HISTÓRICO: LA INDUSTRIA EN LA ÉPOCA DE DATABASE-FIRST FRENTE A LA SITUACIÓN ACTUAL

Para comprender por qué se opta por una arquitectura Domain-First en lugar de Database-First, es útil analizar cómo eran los requisitos tecnológicos cuando el enfoque Database-First se consolidó, y cómo han cambiado desde entonces.

CONTEXTO EN LA ÉPOCA DE DATABASE-FIRST (décadas de 1980-2000):

- Almacenamiento costoso: en los años 80 y 90, el coste por gigabyte de almacenamiento era muy elevado. Esto hacía que la base de datos fuera uno de los recursos más caros y críticos del sistema. Era lógico que el diseño del software girara alrededor de optimizar su uso.

- Aplicaciones pequeñas y monolíticas: la mayoría de las aplicaciones eran de pequeño alcance, con pocas pantallas y poca lógica. El negocio "cabía" en el esquema de base de datos.

- Baja concurrencia: los sistemas atendían a decenas o cientos de usuarios simultáneos, no a miles o millones. Los bloqueos de la base de datos no eran un problema crítico.

- Alta latencia aceptable: los usuarios aceptaban tiempos de respuesta de varios segundos. No había expectativa de respuesta inmediata.

- Sin paralelismo real: los servidores tenían un procesador con un núcleo. No existía la necesidad de diseñar software para ejecución concurrente.

- Despliegue manual e infrecuente: las aplicaciones se desplegaban una o dos veces al año, con paradas planificadas. No había CI/CD (integración y despliegue continuos).

SITUACIÓN ACTUAL (2020-2026):

- Almacenamiento prácticamente gratuito: el coste por terabyte se ha reducido en varios órdenes de magnitud. La base de datos ya no es el recurso limitante del sistema.

- Aplicaciones complejas y distribuidas: las aplicaciones gestionan dominios con múltiples reglas de negocio, integraciones con terceros y requisitos de escalabilidad que no caben en un simple esquema relacional.

- Alta concurrencia: las aplicaciones web atienden a miles o millones de peticiones simultáneas. En un sistema donde la base de datos es el centro, cada transacción puede bloquear filas o tablas, creando cuellos de botella. Según el estudio de Forsgren et al. (2018), las organizaciones con arquitecturas desacopladas tienen frecuencias de despliegue hasta 200 veces superiores y tiempos de recuperación 2.604 veces más rápidos.

- Baja latencia exigida: los usuarios esperan respuestas en milisegundos. Los bloqueos de base de datos por transacciones largas son inaceptables.

- Procesadores multinúcleo y paralelismo: los servidores tienen decenas de núcleos. El software necesita aprovechar la concurrencia (hilos virtuales, programación asíncrona).

- Despliegue continuo (CI/CD): las aplicaciones se despliegan varias veces al día. La rigidez de un esquema de base de datos compartido entre módulos dificulta esta agilidad.

- Cloud computing y elasticidad: los servicios se escalan horizontalmente (más instancias del mismo servicio). Si todos los módulos comparten una única base de datos, esta se convierte en un punto de fallo único que no escala.

¿Qué es una transacción de base de datos?

Una transacción es un grupo de operaciones que deben ejecutarse todas juntas o ninguna. El ejemplo clásico es una transferencia bancaria: hay que restar el dinero de una cuenta Y sumarlo a otra. Si la operación falla a mitad de camino (por ejemplo, se resta de una cuenta pero falla al sumar a la otra), la transacción se "deshace" (rollback) y todo queda como estaba. Esto garantiza que los datos no queden en un estado inconsistente (dinero que desaparece sin llegar a su destino).

En Spring Boot, las transacciones se gestionan automáticamente: Spring abre una transacción al principio de cada operación y la cierra (confirma o deshace) al final, sin que el programador tenga que escribir código para ello.

¿Qué es un bloqueo de base de datos?

Cuando una transacción está modificando una fila de una tabla, la base de datos "bloquea" esa fila temporalmente para que ninguna otra transacción la modifique al mismo tiempo. Es como poner un cartel de "ocupado" en un probador de una tienda: mientras alguien está dentro, los demás tienen que esperar.

Esto funciona bien cuando los bloqueos son cortos y afectan a pocas filas. El problema aparece cuando muchas transacciones intentan acceder a las mismas tablas al mismo tiempo. Cada transacción tiene que esperar a que las demás liberen sus bloqueos, y el rendimiento se reduce.

Qué es un deadlock?

Un deadlock (bloqueo mutuo) ocurre cuando dos transacciones se bloquean entre sí: la transacción A espera a que B libere una fila, y B espera a que A libere otra. Ninguna de las dos puede avanzar. La base de datos detecta esta situación y cancela una de las dos transacciones para desbloquear la otra.

¿Cómo afecta esto al diseño?

En un sistema con un único esquema compartido entre todos los módulos, una transacción en el módulo de pedidos puede bloquear tablas que necesita el módulo de clientes, paralizando operaciones que no tienen nada que ver entre sí.

Con esquemas separados por módulo (como en este proyecto), cada módulo solo bloquea sus propias tablas. Una operación larga en pedidos no afecta a las operaciones de clientes ni de catálogo.

Referencia:

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly.

Capítulo 7 (Transactions) y Capítulo 9 (Consistency and Consensus).

En resumen: Database-First no es un enfoque "anticuado", sino un enfoque diseñado para un contexto tecnológico con requisitos muy diferentes a los actuales. Optar por Domain-First con modularidad no es rechazar Database-First: es adaptar la arquitectura a los requisitos reales del desarrollo de software en 2026.

2. PILAR 1 — ENCAPSULAMIENTO Y AISLAMIENTO DE FALLOS

El principio más básico de la ingeniería del software es el encapsulamiento: cada módulo debe ocultar sus decisiones internas para que los demás no dependan de ellas.Cuando todos los módulos comparten una única base de datos con integridad referencial global, cualquier cambio en la estructura de datos de un módulo puede afectar a los demás de forma inesperada. Por ejemplo, si el módulo de Catálogo necesita cambiar el tipo de dato de una columna, y el módulo de Pedidos tiene una clave foránea (FK) apuntando a esa tabla, el cambio en Catálogo puede romper Pedidos.

Con esquemas separados, cada módulo es dueño de sus tablas. Si el módulo de Catálogo cambia su estructura interna, el módulo de Pedidos no se ve afectado, porque no tiene ninguna dependencia directa a nivel de base de datos.

Referencia fundamental:

Parnas, D. L. (1972). "On the Criteria To Be Used in Decomposing Systems into Modules." Communications of the ACM, 15(12), 1053-1058.

Este artículo, publicado hace más de 50 años, ya establecía que un módulo debe ocultar sus estructuras de datos internas ("Information Hiding"). Compartir una base de datos entre módulos es exactamente lo contrario de este principio: expone la estructura interna de cada módulo a todos los demás.

Caso de éxito — Shopify:

Shopify, una de las plataformas de comercio electrónico más grandes del mundo, pasó de un monolito tradicional a un monolito modular. Forzaron por software que ningún módulo pudiera acceder a las tablas de otro. Según sus ingenieros, esto redujo los incidentes en producción de forma significativa y aumentó la velocidad de despliegue, porque los equipos podían trabajar en sus módulos sin miedo a romper otros.

Fuente: Shopify Engineering Blog — "Deconstructing the Monolith" (2019).

3. PILAR 2 — AGILIDAD DE DESARROLLO Y CARGA COGNITIVA

La "carga cognitiva" es la cantidad de información que un desarrollador necesita tener en la cabeza para poder trabajar en una parte del sistema. En términos sencillos: cuántas cosas tienes que entender antes de poder hacer un cambio.

En un sistema con una base de datos única de, por ejemplo, 50 tablas interconectadas, un desarrollador que quiere modificar el módulo de Clientes necesita entender todo el diagrama entidad-relación, porque cualquier tabla puede tener dependencias con cualquier otra.

Con esquemas separados, el desarrollador de Clientes solo necesita conocer las tablas del esquema "customers" (en este proyecto, una sola tabla). Esto reduce drásticamente el tiempo de incorporación de nuevos miembros al equipo y la probabilidad de errores.

Referencia:

Forsgren, N.; Humble, J.; Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution.

Este libro, basado en años de investigación del programa DORA (DevOps Research and Assessment, respaldado por Google Cloud), documenta con datos de miles de equipos de desarrollo que las organizaciones con arquitecturas desacopladas (donde los módulos pueden desarrollarse, desplegarse y probarse de forma independiente) tienen un rendimiento significativamente superior en las cuatro métricas clave:

- Frecuencia de despliegue
- Tiempo desde el commit hasta producción (lead time)
- Tasa de fallos al desplegar (change failure rate)
- Tiempo medio de recuperación ante fallos (MTTR)

Referencia adicional:

Brooks, F. P. (1975). The Mythical Man-Month. Addison-Wesley.

Brooks observó que la complejidad de un sistema crece de forma más que proporcional con el número de interconexiones. Las claves foráneas globales son exactamente esas interconexiones. La modularidad las mantiene bajo control.

Caso de éxito — DoorDash:

DoorDash documentó que, al modularizar su sistema y separar las responsabilidades de datos, el tiempo de incorporación (onboarding) de un nuevo desarrollador se redujo de semanas a días, porque el programador ya no tenía que aprender todo el modelo de datos global antes de hacer su primer cambio productivo.

4. PILAR 3 — INTEGRIDAD LÓGICA FRENTE A RIGIDEZ FÍSICA

Una objeción habitual a este diseño es: "Si no hay claves foráneas entre módulos, ¿cómo se garantiza la integridad de los datos?"

La respuesta es que la integridad se mantiene de dos maneras complementarias:

a) DENTRO de cada módulo, SÍ existen claves foráneas normales. Ejemplo: en el schema "orders", la tabla order_line tiene una FK hacia purchase_order. Si se intenta insertar una línea de pedido con un order_id que no existe, la base de datos lo rechaza.

b) ENTRE módulos, las referencias son lógicas (IDs sin FK declarada). Ejemplo: purchase_order guarda un customer_id (UUID del cliente), pero no hay una FK declarada hacia customers.customer. La consistencia se garantiza en la capa de aplicación: el servicio que crea un pedido puede verificar que el cliente existe antes de crear el pedido.

Esto sigue el principio de Responsabilidad Única (SRP) aplicado a los datos: un esquema de base de datos debe tener una sola razón para cambiar. Si la tabla de Pedidos cambia porque cambió el sistema de Clientes, se está violando este principio.

Referencia:

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Martin describe los conceptos de "Rigidez" (difícil de cambiar), "Fragilidad" (un cambio pequeño rompe cosas lejanas) e "Inmovilidad" (imposible reutilizar partes). Una base de datos con claves foráneas globales entre todos los módulos tiende a ser rígida, frágil e inmóvil.

Nota importante:

No se dice que la integridad referencial sea incorrecta. Se dice que es "local al contexto". La integridad existe dentro del módulo, pero la relación entre módulos se gestiona mediante identificadores y lógica de aplicación. Esto es exactamente lo que propone Evans (2003) con el concepto de "Aggregate": las reglas de consistencia fuerte se aplican dentro de los límites del aggregate, no entre aggregates de distintos bounded contexts.

Caso de éxito — Segment (Twilio):

Segment comenzó con una base de datos centralizada y compartida entre todos sus módulos. A medida que crecieron, los cambios en una tabla provocaban fallos en partes del sistema aparentemente no relacionadas. Tras modularizar y separar las responsabilidades de datos, la estabilidad del servicio mejoró significativamente.

Fuente: Segment Engineering Blog.

5. PILAR 4 — AUTONOMÍA DE EQUIPOS Y LA LEY DE CONWAY

En 1967, Melvin Conway formuló una observación que se ha convertido en una de las leyes más citadas de la ingeniería del software:

"Las organizaciones que diseñan sistemas están limitadas a producir diseños que son copias de sus estructuras de comunicación."

— Conway, M. E. (1968). "How Do Committees Invent?"

En términos prácticos: si un equipo de desarrollo está organizado por funcionalidades (un grupo trabajando en el módulo de Inventario, otro en Ventas), pero todos comparten una única base de datos, cada cambio técnico requerirá una reunión de sincronización entre equipos. La base de datos compartida se convierte en un cuello de botella organizativo, no solo técnico.Con esquemas separados, cada equipo (o cada desarrollador, en un proyecto más pequeño) puede trabajar en su módulo sin coordinación constante con los demás.

Caso de éxito — Spotify:

Spotify popularizó el modelo de "Squads" (escuadrones), donde cada equipo pequeño es autónomo y es dueño de su código y sus datos. Para que esto funcione, cada squad necesita tener control total sobre su esquema de datos. Si tuvieran que pedir permiso a otros equipos para cambiar una tabla compartida, la velocidad de innovación se detendría.

Fuente: Kniberg, H. & Ivarsson, A. (2012). "Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds."

Caso de éxito — Amazon (El "Mandato de Bezos"):

En 2002, Jeff Bezos emitió un mandato interno que se ha hecho famoso en la industria del software: "Todos los equipos expondrán sus datos y funcionalidades a través de interfaces de servicio (APIs). No se permite ninguna otra forma de comunicación: ni lectura directa de bases de datos ajenas, ni memoria compartida." Este mandato prohibió explícitamente que un equipo accediera a la base de datos de otro equipo, forzando la modularidad. Amazon pasó de ser una librería web a ser el líder mundial en computación en la nube (AWS), y esta separación de datos fue uno de los factores clave. Fuente: Yegge, S. (2011). "Stevey's Google Platforms Rant" (testimonio interno).

6. PILAR 5 — PERSISTENCIA POLÍGLOTA

En los años 90, la base de datos relacional era prácticamente la única opción para almacenar datos. Hoy existen muchos tipos de bases de datos, cada una optimizada para un tipo de problema diferente:

- Bases de datos relacionales (PostgreSQL, MySQL): buenas para datos estructurados con relaciones complejas y transacciones.

- Bases de datos de documentos (MongoDB): buenas para datos semi-estructurados y esquemas flexibles.

- Bases de datos clave-valor (Redis): buenas para caché y datos de sesión con baja latencia.

- Bases de datos de grafos (Neo4j): buenas para relaciones complejas entre entidades.

- Bases de datos de series temporales (InfluxDB): buenas para métricas y logs.

Esto se conoce como "Persistencia Políglota" (Polyglot Persistence):

Fowler, M. "Polyglot Persistence." martinfowler.com. "Cualquier organización de tamaño medio tendrá una mezcla de tecnologías de almacenamiento de datos."

Si todo el sistema está atado a un único esquema relacional con integridadreferencial global, no hay forma de que un módulo use una tecnología de almacenamiento diferente sin reescribir el sistema completo.

Con la arquitectura modular de este proyecto, si en el futuro fuera necesario que el módulo de reports usara una base de datos diferente (por ejemplo, un sistema optimizado para consultas analíticas), el cambio solo afectaría a ese módulo.

Caso de éxito — Netflix:

Netflix gestiona más de mil microservicios. Cada uno es dueño de sus datos: el historial de visionado se guarda en Cassandra (NoSQL), la facturación usa bases de datos relacionales, y las recomendaciones usan sistemas especializados.

Si Netflix usara una base de datos única, un error en el servicio de recomendaciones podría bloquear las tablas de usuarios e impedir que alguien iniciara sesión. Al aislar los datos por módulo, el sistema es resiliente: si falla un servicio, el resto sigue funcionando.

Fuente: Netflix Technology Blog.

Caso de éxito — eBay:

eBay fue una de las primeras empresas en documentar públicamente que su base de datos centralizada se convirtió en el principal cuello de botella para escalar.

El fenómeno de "Database Contention" (competencia por acceso a las mismas tablas) les obligó a separar los datos por dominio funcional.

7. EL MÓDULO REPORTS Y EL PRINCIPIO CQRS

El módulo reports es un caso especial que ilustra una ventaja concreta de la modularidad. Este módulo no sigue DDD completo: no tiene capa de dominio, no tiene value objects propios. Está diseñado exclusivamente para consultas de reporting (lecturas agregadas: top productos más vendidos, top clientes por volumen de compra).

¿Qué es CQRS?

CQRS son las siglas de "Command Query Responsibility Segregation", que en español significa "Separación de Responsabilidades entre Comandos y Consultas". La idea es sencilla: separar el modelo que usamos para escribir datos (crear pedidos, registrar clientes) del modelo que usamos para leer datos (generar informes, consultar estadísticas). Cada modelo se optimiza para su propósito.

En este proyecto, CQRS se aplica de forma parcial:

- Los módulos de escritura (catalog, customers, orders) usan DDD completo.

- El módulo de lectura (reports) usa una única tabla denormalizada (reports.order_report) que contiene todos los datos necesarios para los informes en una sola fila, evitando la necesidad de hacer JOINs entre tablas de distintos módulos.

¿Cómo se alimenta la tabla de reports?

Cada vez que se confirma un pedido (caso de uso set_order_as_confirmed en el módulo orders), se dispara un evento (OrderConfirmedEvent) a través de la interfaz OrderConfirmed definida en el shared_kernel. El módulo reports implementa esa interfaz y recibe los datos del pedido confirmado, insertándolos en la tabla order_report como un snapshot inmutable.

¿Por qué una tabla denormalizada y no JOINs entre esquemas? Una objeción habitual es: "¿Cómo haces un informe que necesite datos de dos módulos si no tienes JOINs ni claves foráneas entre ellos?" Existen tres estrategias posibles, de menor a mayor complejidad:

a) Composición en la capa de aplicación:

El servicio de informes llama internamente a los módulos de clientes y pedidos (via internal_api del shared_kernel) para obtener los datos necesarios, y los combina en un DTO de respuesta.

b) Vistas de base de datos:

Se puede crear una vista SQL que cruce esquemas para lectura, sin afectar a las tablas de escritura.

c) Read Model denormalizado (CQRS — la opción usada en este proyecto): Se mantiene una tabla separada con los datos ya "pre-cocinados" para los informes. Cada vez que ocurre un evento de escritura relevante (como confirmar un pedido), los datos se replican a esta tabla de reporting.

La opción (c) es la que usa este proyecto y es la recomendada por la literatura: Young, G. (2010). "CQRS Documents." Greg Young es quien formalizó el patrón CQRS.

Fowler, M. "Reporting Database." martinfowler.com.

Fowler recomienda explícitamente usar una base de datos separada para reporting. Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly. En el capítulo dedicado a reporting, Newman describe el acceso directo a la base de datos de otro servicio para reportes como un "anti-patrón de acoplamiento" que impide la evolución del software.

Ventajas concretas de este enfoque:

- Un informe pesado no bloquea las operaciones de venta.

- Si el volumen de datos crece, la tabla de reporting se puede migrar a un sistema analítico sin tocar el código de ventas.

- Cada módulo sigue siendo independiente y puede evolucionar sincoordinación.

8. EVIDENCIA ACADÉMICA

Además de los casos de éxito de la industria, existen estudios académicos que respaldan este enfoque:

MacCormack, A. et al. (Harvard Business School / MIT). "Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and Proprietary Code." Conclusión: la modularidad actúa como una "opción financiera" — permite cambiar una parte del sistema sin pagar el coste de reconstruir el resto. Los sistemas con menor acoplamiento entre módulos son más fáciles y baratos de mantener.

Association for Computing Machinery (ACM) — estudios sobre mantenimiento de sistemas heredados: indican que en sistemas con bases de datos compartidas, una proporción muy significativa del esfuerzo de mantenimiento se gasta en arreglar efectos colaterales de cambios que no deberían haber afectado a otros módulos.

Caso de éxito a pequeña escala — Basecamp (37signals):

Basecamp, la empresa que creó Ruby on Rails, defiende el monolito modular como la arquitectura más adecuada para la mayoría de aplicaciones. David Heinemeier Hansson (DHH), su creador, sostiene que la clave no es la distribución de servicios, sino la organización interna del código. Fuente: DHH. "The Majestic Monolith" (2016). Signal v. Noise (Basecamp).

Caso de éxito a pequeña escala — LinkedIn: LinkedIn tenía una base de datos centralizada llamada "Leo". Llegó un punto en que no podían añadir nuevas funcionalidades porque el grafo de dependencias de la base de datos era tan complejo que nadie se atrevía a tocarlo. Tuvieron que detener el desarrollo de producto para modularizar y separar sus bases de datos (Proyecto Inversion).

9. ¿POR QUÉ NO MICROSERVICIOS?

Si se van a tener módulos independientes, una pregunta legítima es: ¿por qué no hacer microservicios directamente?

Nota: no tengo experiencia directa con microservicios. Lo que describo aquí proviene de las fuentes que he consultado durante el desarrollo de este proyecto (libros como "Building Microservices" de Sam Newman y "Microservices Patterns" de Chris Richardson,y documentación de Spring). Este proyecto es un monolito modular, y las referencias a microservicios son para contextualizar las decisiones y entender los principios, no para describir experiencia práctica.

Los microservicios, según estas fuentes, añaden una complejidad operativa considerable (despliegue distribuido, comunicación por red, monitorización de muchos servicios, gestión de fallos cuando un servicio no responde) que no está justificada para un proyecto de fin de grado ni para muchos proyectos reales de tamaño medio.

El monolito modular es un punto intermedio razonable:

- Mantiene la simplicidad operativa (un único .jar, un único despliegue).

- Aplica los principios de bajo acoplamiento y alta cohesión.

- Si en el futuro fuera necesario extraer un módulo a un microservicio, los límites ya están definidos y el esfuerzo sería mucho menor.

Referencia:

Richardson, C. (2018). Microservices Patterns. Manning.

Heinemeier Hansson, D. (2016). "The Majestic Monolith."

10. CONCLUSIÓN

La decisión de organizar este proyecto como un monolito modular con esquemasseparados por bounded context se basa en principios bien establecidos:

✓ Encapsulamiento (Parnas, 1972): cada módulo oculta su estructura interna.

✓ Bajo acoplamiento (Forsgren et al., 2018): los módulos son independientes.

✓ Integridad donde es necesaria: FKs dentro de cada módulo, referencias lógicas entre módulos.

✓ Preparación para el futuro: persistencia políglota posible si es necesario.

✓ Reporting optimizado (CQRS): modelo de lectura separado e independiente.

✓ Autonomía de equipos (Conway, 1968): cada equipo trabaja sin bloquear a los demás.

Estas decisiones están respaldadas por la bibliografía de referencia, por estudios académicos y por la práctica de empresas de todos los tamaños, desde startups como Basecamp y Segment, hasta empresas como Shopify, Amazon, Netflix y Spotify.

No se trata de que un enfoque sea "mejor" o "peor". Se trata de elegir el enfoque más adecuado para el contexto de este proyecto, y de justificar esaelección con argumentos sólidos y verificables.

BIBLIOGRAFÍA DE ESTE APARTADO

- Brooks, F. P. (1975). The Mythical Man-Month. Addison-Wesley.

- Conway, M. E. (1968). "How Do Committees Invent?" Datamation, 14(5), 28-31.

- Evans, E. (2003). Domain-Driven Design. Addison-Wesley.

- Forsgren, N.; Humble, J.; Kim, G. (2018). Accelerate. IT Revolution.

- Fowler, M. "CQRS." https://martinfowler.com/bliki/CQRS.html

- Fowler, M. "Polyglot Persistence." https://martinfowler.com/bliki/PolyglotPersistence.html

- Fowler, M. "Reporting Database." https://martinfowler.com/bliki/ReportingDatabase.html

- Fowler, M. "Shared Database." https://martinfowler.com/bliki/SharedDatabase.html

- Heinemeier Hansson, D. (2016). "The Majestic Monolith." Signal v. Noise.

- Kniberg, H. & Ivarsson, A. (2012). "Scaling Agile @ Spotify."

- MacCormack, A. et al. Harvard / MIT. "Exploring the Structure of Complex Software Designs."

- Martin, R. C. (2017). Clean Architecture. Prentice Hall.

- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly.

- Parnas, D. L. (1972). "On the Criteria To Be Used in Decomposing Systems into Modules." Communications of the ACM, 15(12).

- Richardson, C. (2018). Microservices Patterns. Manning.

- Vernon, V. (2013). Implementing Domain-Driven Design. Addison-Wesley.

- Young, G. (2010). "CQRS Documents."

3. Conclusiones y recomendaciones

Conclusiones personales del proyecto, alcance de objetivos, mejoras futuras.

4. Bibliografía y webgrafía

Bibliografía en formato APA:

Apellido, N. (año). *Título del trabajo* (edición). Editorial.

Ejemplo:

Sáenz de Jubera Ocón, M., & Alonso Ruíz, R. A. (Coords.). (2021). Ocio y educación: experiencias, innovación y transferencia. Universidad de La Rioja.

Webgrafía de las principales páginas, tutoriales, videos, etc. consultados para completar el proyecto.

Título de la página. (Mes y año del momento de consulta). Nombre de la página web. URL

Ejemplo:

The countdown: A prophecy, crowds and a TikTok takedown. (febrero 2025). BBC News. https://www.bbc.com/news/election-us-2020-54596667

Página oficial de MySql (febrero 2025). https://www.mysql.com/

Ejemplos de contenidos:

Las tablas se enuncian de la siguiente manera (título y pie):

Tabla c. Cronograma de actuación para el proyecto de investigación.

Actividad	Tiempo							
	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5	Mes 6	Mes 7	Mes 8

Fuente: Elaboración propia.

Las figuras y las fotografías se enuncian de la siguiente forma (título y pie):

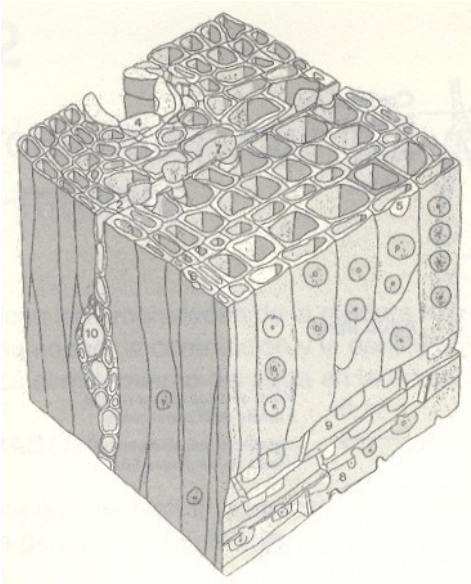


Figura 1. Estructura anatómica de la madera de coníferas

Fuente: Manual Técnico de secado de maderas. Fernández-Golfín & Conde García.